UNIVERSIDADE ESTADUAL DA PARAÍBA - UEPB

DISCIPLINA: DESENVOLVIMENTO WEB (BACK-END)

PROF.: ANDRÉ ALMEIDA

DISCENTE: LUCAS MARTINS FERNANDO

CURSO: ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Atividade 4

Conceitos de Node.js e utilização de Express para construção de APIs

1. Analise o código a seguir:

```
const alunos = [
    { id: 1, nome: "Ana", nota: 8 },
    { id: 2, nome: "Carlos", nota: 5 },
    { id: 3, nome: "Marina", nota: 9 }
];

const aprovados = alunos.filter(aluno => aluno.nota >= 7);

const nomes = aprovados.map(aluno => aluno.nome);

console.log(nomes);
```

a) Qual será a saída do programa?
Resposta:
['Ana', 'Marina']

b) Explique a diferença entre os métodos filter() e map().
O filter() vai filtrar os elementos e criar um novo array contendo os elementos conforme as regras que escolhemos, nesse caso um objeto chave:valor . Já o método map() vai transformar esse objeto em Strings conforme a regra que decidimos que é const nomes = aprovados.map(aluno => aluno.nome); que faz entrar o objeto chave:valor e sair só a string nome.

c) Modifique o código para obter somente os alunos com nota inferior a 7.

```
const alunos = [
    { id: 1, nome: "Ana", nota: 8 },
    { id: 2, nome: "Carlos", nota: 5 },
    { id: 3, nome: "Marina", nota: 9 }
];

const aprovados = alunos.filter(aluno => aluno.nota >= 7);
const reprovados = alunos.filter(aluno => aluno.nota < 7);
```

const nomes = aprovados.map(aluno => aluno.nome);
const nomesReprovados = reprovados.map(aluno => aluno.nome);
console.log(`Aprovados: ${nomes}, Reprovados: ${nomesReprovados}`);

d) Explique por que o uso de const não impede a alteração dos elementos de um array.
O const vai congelar a váriavel para que ela sempre esteja no mesmo endereço de memória, mas os elementos dentro dela não, eles podem sofrer alteração.

2. Responda aos itens a seguir.

a) O que é Node.js e qual é sua relação com o motor JavaScript V8?
O Node.js vai ser o responsável por trabalhar o JavaScript, ele tem diversas funções, permite a conexão de bancos de dados, trabalhar arquivos, e criar os servidores web. a relação seria que o V8 funciona como um tradutor ajudando na linguagem de máquina.

b) Qual é a diferença entre executar JavaScript no navegador e executá-lo com Node.js?
No navegador o javascript atua de forma limitada, já no Node.js ele se torna totalmente amplo, ele consegue acessar arquivos do seu computador, ler o terminal, conectar em bancos de dados e criar servidores web (Back-end).

c) Qual é a finalidade do npm?
Funciona como um tipo de gerenciador e pacotes, se precisamos de um código para o servidor podemos baixar por ele como se fosse um cardápio.

d) Qual é a função dos arquivos package.json e package-lock.json?
O package.json funciona como a identidade do projeto. Ele guarda o nome do projeto, o autor, e principalmente: a lista de quais dependências o projeto precisa para funcionar. O package-lock.json serve para travar as versões exatas dos pacotes. Isso existe para garantir que, se outro programador rodar o seu código, o npm vai baixar a versão correta.

e) Diferencie uma dependência de produção de uma dependência de desenvolvimento.
Dependência de produção são os pacotes essenciais necessários para manter o código rodando online no servidor. Já de desenvolvimento são os pacotes que o programador precisa quando estiver desenvolvendo a aplicação.

f) Explique a diferença entre importar um módulo nativo do Node.js e importar um pacote instalado pelo npm.
Um módulo nativo do Node.js já vem embutido na instalação padrão da ferramenta, está pronto para uso sem a necessidade de downloads. Já um pacote do npm é um código criado pela comunidade que não vem de fábrica, sendo obrigatório baixá-lo da internet para o projeto usando o comando npm install antes de poder utilizá-lo.

3. Considere o código:

```
const http = require("http");
```

```
const servidor = http.createServer((req, res) => {
   res.end("Servidor ativo");
});

servidor.listen(3000);
```

a) Explique a função de require("http").
Essa função serve para importar o módulo nativo do http para dentro do nosso arquivo.

b) Qual é a finalidade de createServer()?
Serve para criar o nosso servidor web.

c) O que representam os parâmetros req e res?
O req é a requisição(request), que é o pedido ou os dados que chegam do cliente. O res será a resposta(response), que é o que o servidor vai devolver para quem fez o pedido.

d) Para que serve listen(3000)?
Serve para ligar o servidor e fazer ele ficar esperando as requisições na porta 3000 do computador.

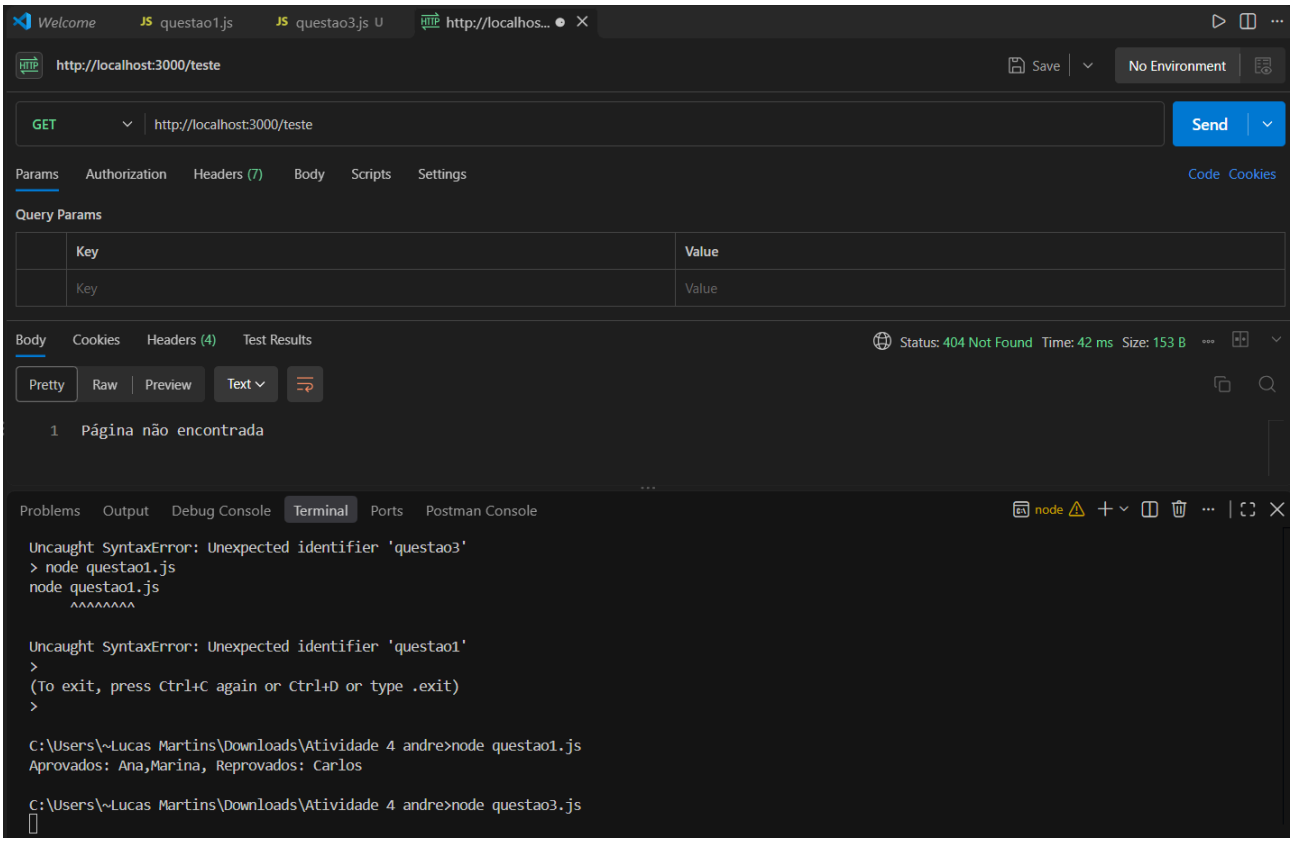
e) Modifique o código para responder com a mensagem "Página inicial" quando o caminho solicitado for / e com "Página não encontrada" nos demais caminhos.

```
const http = require("http");

const servidor = http.createServer((req, res) => {
   // Se a URL digitada for exatamente a barra (página inicial)
   if (req.url === "/") {
      res.end("Página inicial");
   } else {
      // Se for qualquer outra rota, muda o status para 404 e avisa que não encontrou
      res.statusCode = 404;
      res.end("Página não encontrada");
   }
});

servidor.listen(3000);
```

f) Altere a implementação para retornar o status HTTP 404 quando a rota não existir.



4. Um desenvolvedor deseja criar uma API usando Express.js, mas ainda não possui um projeto configurado.

a) Apresente os comandos necessários para criar uma pasta de projeto, inicializar o npm e instalar o Express.js.

1. **mkdir meu-projeto (cria a pasta do projeto)**
2. **cd meu-projeto (entra na pasta)**
3. **npm init -y (cria o arquivo package.json rapidamente)**
4. **npm install express (baixa e instala o pacote do Express)**

b) Escreva o código mínimo para criar um servidor Express na porta 3000.

```
const express = require("express");
const app = express();
// Ligando o servidor na porta 3000
app.listen(3000, () => {
   console.log("Servidor Express rodando na porta 3000");
});
```

c) Implemente uma rota GET / que responda com um objeto JSON contendo uma mensagem de boas-vindas.

```
const express = require("express");
const app = express();

// Rota GET inicial
```

```
app.get("/", (req, res) => {
   res.json({ mensagem: "Bem-vindo à nossa API!" });
});

// Ligando o servidor na porta 3000
app.listen(3000, () => {
   console.log("Servidor Express rodando na porta 3000");
});
```

d) Explique a diferença entre res.send() e res.json().
res.send() é mais genérico: ele tenta adivinhar o que você está enviando (texto, HTML, etc.) e manda do jeito que achar melhor. Já o res.json() é focado: ele converte tudo para o formato JSON certinho e avisa para o cliente (no cabeçalho da resposta) que o dado que está chegando é estritamente um JSON.

e) O que acontece quando uma requisição chega a uma rota que não foi definida e a aplicação não possui um tratamento específico para ela?
O próprio Express percebe que a rota não existe e devolve automaticamente o status 404 (Not Found), mostrando uma página HTML bem feia no navegador com a mensagem padrão, tipo: Cannot GET /rota-inventada.

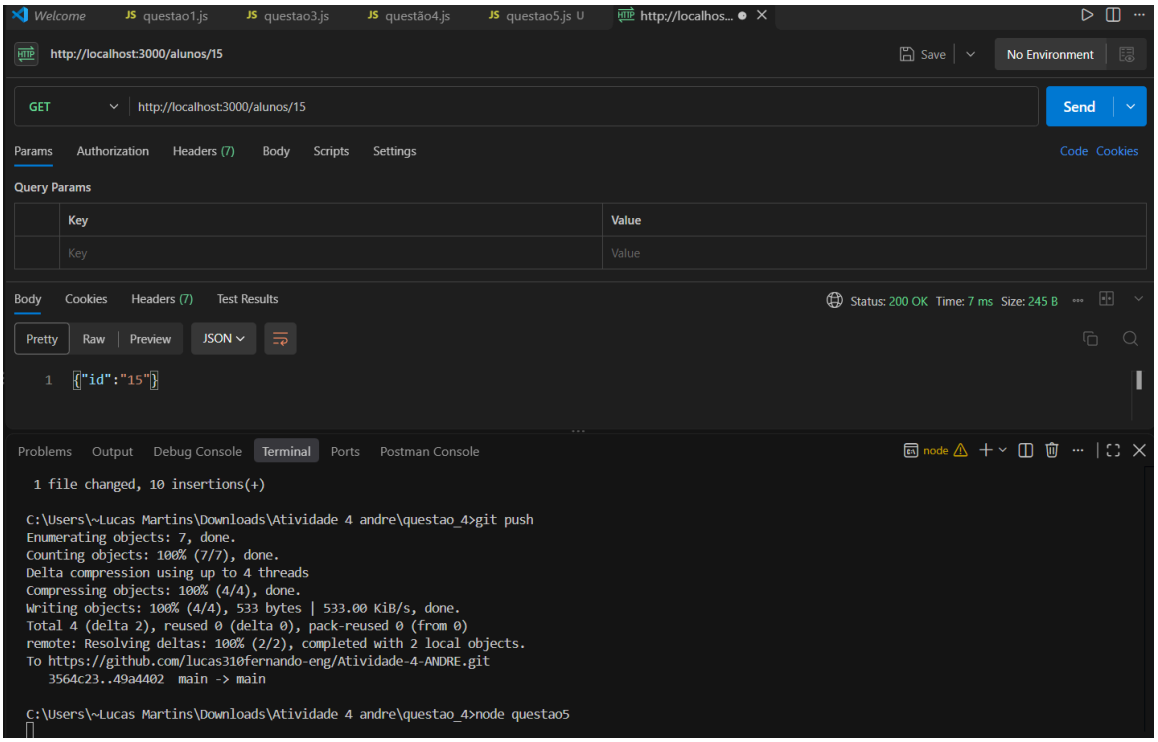
5. Considere o seguinte código:

```
const express = require("express");
const app = express();

app.get("/alunos/:id", (req, res) => {
   res.json({
      id: req.params.id
   });
});

app.listen(3000);
```

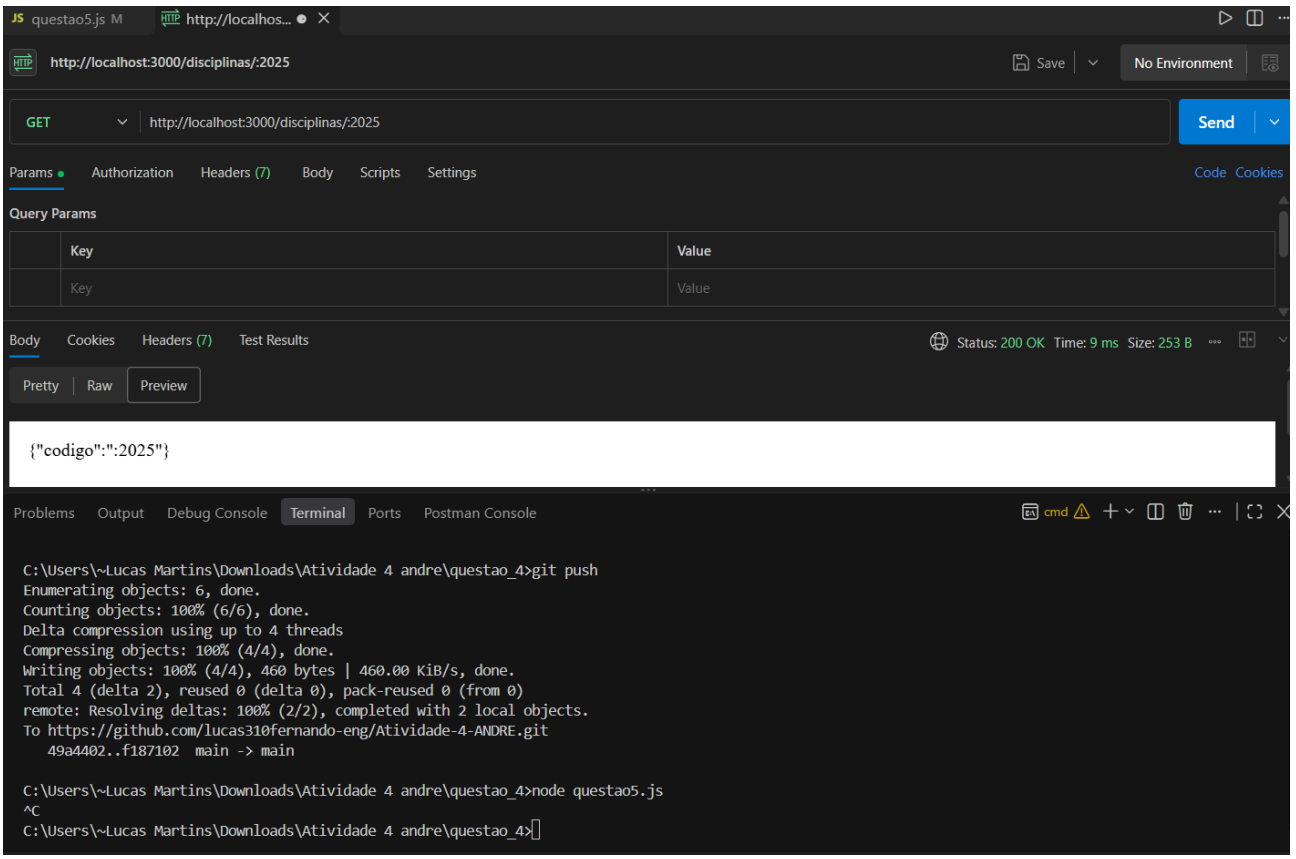
a) Qual será a resposta para uma requisição GET /alunos/15?
{"id": "15"}



b) Qual é a finalidade de req.params?

Ele serve para capturar os parâmetros dinâmicos que o usuário digita direto na URL. No código do exercício, ele é o responsável por pegar o valor que foi colocado no lugar do :id.

c) Implemente uma rota GET /disciplinas/:codigo que retorne o código da disciplina recebido na URL.



d) Explique a diferença entre uma rota com o caminho /alunos/:id e uma rota com o caminho /alunos.
A rota /alunos/:id é dinâmica, ela exige que você passe um valor na frente da barra (ex: /alunos/10) e geralmente serve para buscar apenas um aluno específico. Já a rota /alunos é estática e normalmente é usada para listar todos os alunos de uma vez.

e) O valor de req.params.id é recebido originalmente como número ou string? Explique.
É recebido originalmente como string (texto). Isso acontece porque tudo o que é digitado e trafegado na URL da internet é tratado como texto. Se a gente precisar fazer um cálculo ou buscar em um banco de dados usando esse ID como número, precisamos converter ele no código usando funções como o Number() ou parseInt().

6. Considere as requisições abaixo:

• GET /alunos?curso=Computacao&semestre=3
• POST /alunos, enviando o JSON:
	{
	"nome": "Pedro",
	"curso": "Computação",
	"semestre": 3
	}

a) Como acessar os valores curso e semestre da primeira requisição no Express.js?
Usamos o req.query. No código ficaria: req.query.curso e req.query.semestre.

b) Como acessar os dados enviados no corpo da segunda requisição?
Usamos o req.body. Para acessar os dados específicos, usamos: req.body.nome, req.body.curso e req.body.semestre.

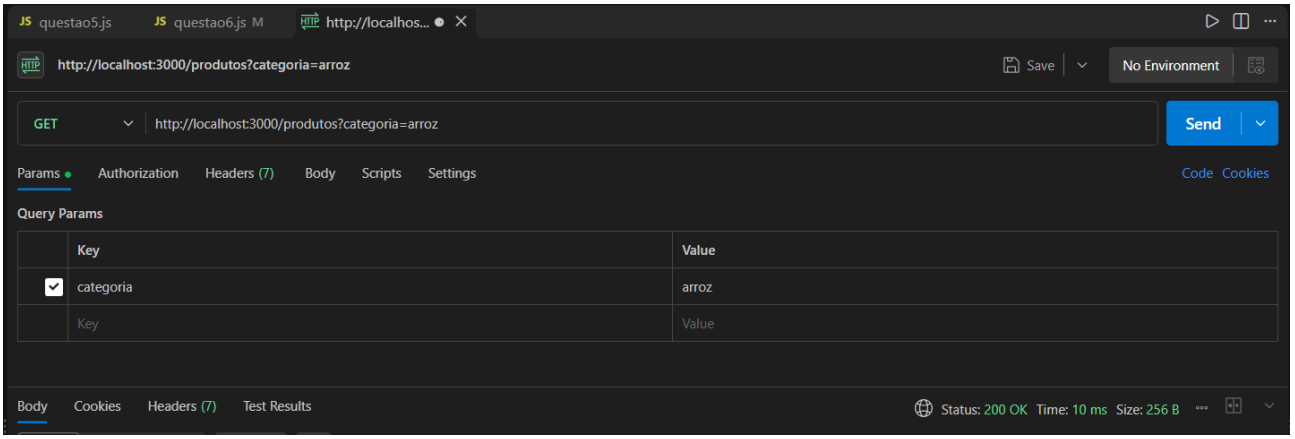
c) Qual middleware deve ser configurado para que o Express.js interprete corpos de requisições JSON?
Precisamos configurar o middleware express.json(). Fazemos isso colocando a linha app.use(express.json()); no código do servidor, antes de criar as rotas.

d) Explique a diferença entre req.params, req.query e req.body.
- **req.params: Pega os valores variáveis definidos direto no caminho da URL (ex: /alunos/:id).**
- **req.query: Pega os valores de busca passados no final da URL, logo após o ponto de interrogação (ex: /alunos?curso=Computacao).**
- **req.body: Pega os dados enviados "escondidos" no corpo da requisição, muito usado para enviar formulários e arquivos em métodos como POST e PUT, sem mostrar nada na URL.**

e) Implemente uma rota GET /produtos que receba um parâmetro de consulta chamado categoria e retorne essa categoria em JSON.
const express = require("express");
const app = express();

```
app.listen(3000);

app.get("/produtos", (req, res) => {
   res.json({
      categoria: req.query.categoria
   });
});
```



f) Implemente uma rota POST /produtos que receba nome e preço no corpo da requisição e retorne os dados recebidos.

```
app.post("/produtos", (req, res) => {
   res.json({
      nome: req.body.nome,
      preco: req.body.preco
   });
});
```

7. Analise o código:

```
const express = require("express");
const app = express();

app.use((req, res, next) => {
   console.log(`${req.method} ${req.url}`);
   next();
});

app.get("/", (req, res) => {
   res.send("Página inicial");
});

app.listen(3000);
```

a) O que é um middleware no Express.js?

Um middleware é uma função que fica no "meio do caminho" entre a requisição que chega e a resposta do servidor. Ele serve para interceptar a requisição, podendo alterar dados, fazer verificações (como checar se o usuário está logado) ou registrar informações antes de deixar a requisição seguir para a rota final.

b) Qual é a função de next() nesse código?
A função next() avisa ao Express que o middleware já terminou o seu trabalho e que o servidor pode passar a requisição para frente (para o próximo middleware ou para a rota final).

c) O que aconteceria se o middleware não chamasse next() nem enviasse uma resposta?
A requisição ficaria "pendurada" (carregando infinitamente) no navegador ou no Postman. O cliente ficaria esperando uma resposta que nunca chegaria, até dar erro por tempo limite (timeout).

d) Implemente um middleware que adicione o horário da requisição ao objeto req.

```
const express = require("express");
const app = express();

app.use((req, res, next) => {
   console.log(`${req.method} ${req.url}`);
   next();
});

app.get("/", (req, res) => {
   res.send("Página inicial");
});

app.listen(3000);

app.use((req, res, next) => {
   req.horario = new Date(); // Cria a data/hora atual e guarda dentro de 'req'
   next(); // Passa para a próxima etapa
});
```

e) Explique a diferença entre um middleware global, um middleware aplicado a uma rota específica e um middleware de tratamento de erros.
Global: É configurado com app.use() (como na questão) e roda em todas as requisições e rotas da aplicação.
Específico de rota: É colocado diretamente dentro de uma rota específica (ex: app.get("/painel", meuMiddleware, (req, res) => {...})) e roda apenas quando essa rota for chamada.
Tratamento de erros: É um middleware especial que obrigatoriamente recebe 4 parâmetros (err, req, res, next). Ele só é executado quando algum erro acontece em outra parte do código, impedindo que o servidor quebre (crash).

8. Considere a implementação:

```
app.get("/alunos/:id", (req, res) => {
   const aluno = alunos.find(
      aluno => aluno.id === Number(req.params.id)
   );
```

```
  res.json(aluno);
});
```

a) Qual problema pode ocorrer quando o aluno solicitado não existe?
O método find retorna undefined se não achar o aluno. O Express vai enviar uma resposta vazia com status 200 (sucesso), o que é errado e confunde o cliente (Front-end), pois parece que deu tudo certo quando na verdade o aluno não existe.

b) Modifique o código para retornar o status 404 e uma mensagem JSON quando o aluno não for encontrado.

```
app.get("/alunos/:id", (req, res) => {
  const aluno = alunos.find(aluno => aluno.id === Number(req.params.id));
  if (!aluno) {
    return res.status(404).json({ mensagem: "Aluno não encontrado" });
  }
  res.json(aluno);
});
```

c) Qual é a finalidade de um bloco try...catch?
Evitar que o servidor "quebre" (crash) quando ocorre um erro inesperado. Ele tenta rodar o código (try) e, se der erro, captura esse erro (catch) para que você possa enviar uma resposta amigável ao usuário sem desligar o servidor.

d) Implemente um middleware de tratamento de erros no Express.js, utilizando os parâmetros err, req, res e next.

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ erro: "Erro interno no servidor" });
});
```

e) Por que não é adequado retornar detalhes internos da aplicação, como caminhos de arquivos ou rastreamentos completos de erros, em respostas destinadas aos clientes?
É um risco grave de segurança. Hackers podem usar caminhos de pastas do seu computador, versões de pacotes e detalhes do banco de dados para descobrir vulnerabilidades e invadir a aplicação.

9. Uma API de gerenciamento de alunos começou a crescer. Atualmente, todas as rotas, regras de negócio e dados estão implementados em um único arquivo index.js.

a) Explique duas dificuldades que podem surgir com essa organização.
O arquivo fica gigante e difícil de ler (código espaguete), e gera muitos conflitos (merge conflicts) no GitHub quando vários desenvolvedores tentam editar o mesmo arquivo ao mesmo tempo.

b) Proponha uma estrutura de diretórios contendo, pelo menos, arquivos ou pastas para rotas, controladores, serviços e configuração do servidor.
/src
/routes
/controllers
/services
server.js

c) Explique a responsabilidade de cada um desses componentes.
routes: Apenas define as URLs (caminhos) e aponta para o controlador correto.

controllers: Recebe os dados da requisição (req), repassa para o serviço e envia a resposta (res).

services: Onde fica a regra de negócio real (cálculos, validações pesadas, acesso ao banco de dados).

configuração (server.js): Liga o Express, configura as portas e aplica middlewares globais.

d) Por que separar as rotas da lógica de negócio pode facilitar a manutenção e os testes?
Porque você consegue testar as regras de negócio isoladamente nos services, sem precisar simular requisições HTTP falsas. Além disso, se precisar mudar o Express para outro framework no futuro, a regra de negócio (service) continua intacta.

e) Explique a diferença entre organizar um projeto em módulos e instalar dependências externas.
Organizar em módulos é dividir o seu próprio código em vários arquivos menores dentro do seu projeto. Instalar dependências externas é baixar pacotes prontos feitos por terceiros na internet usando o npm install.

10. [MINI PROJETO PRÁTICO]
Desenvolva uma API REST utilizando Node.js e Express.js para gerenciar um catálogo de jogos eletrônicos. Os dados devem ser armazenados inicialmente em um array, sem utilizar banco de dados.
Cada jogo deve possuir os seguintes atributos: id, titulo, plataforma, genero e anoLancamento.
A API deverá implementar as seguintes funcionalidades:

Método HTTP	Endpoin	Funcionalidade
GET	/jogos	Listar todos os jogos cadastrados
GET	/jogos/:id	Consultar um jogo pelo ID
POST	/jogos	Cadastrar um novo jogo
PUT	/jogos/:id	Atualizar todos os dados de

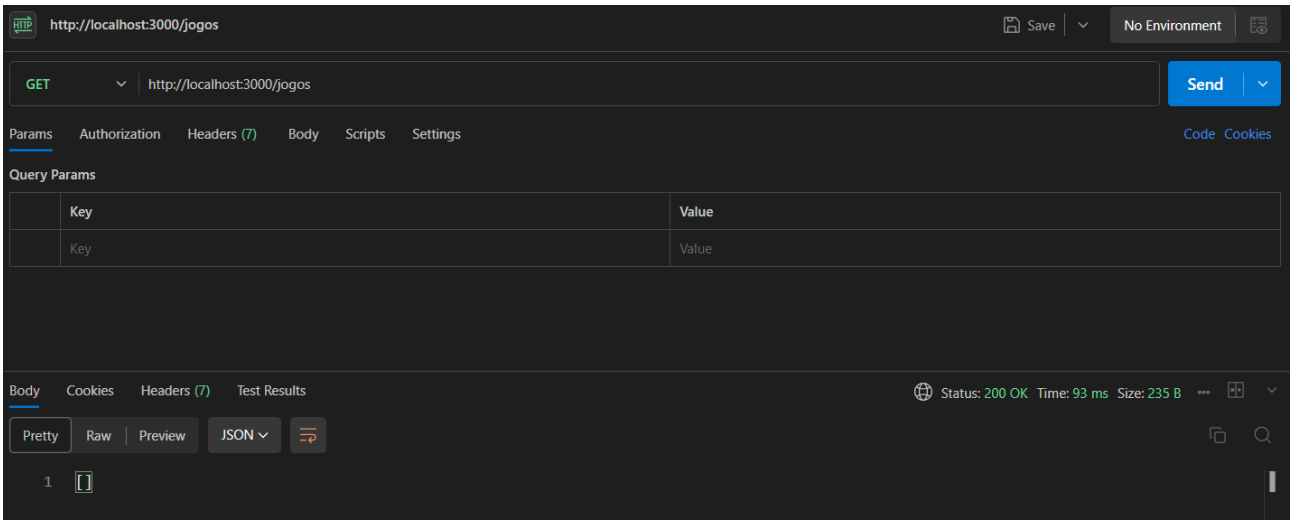
		um jogo
DELETE	/jogos/:id	Remover um jogo do catálogo

a) Configure o projeto utilizando npm e instale o Express.js como dependência.

b) Crie um servidor que escute na porta 3000 e configure o recebimento de requisições com corpo JSON.

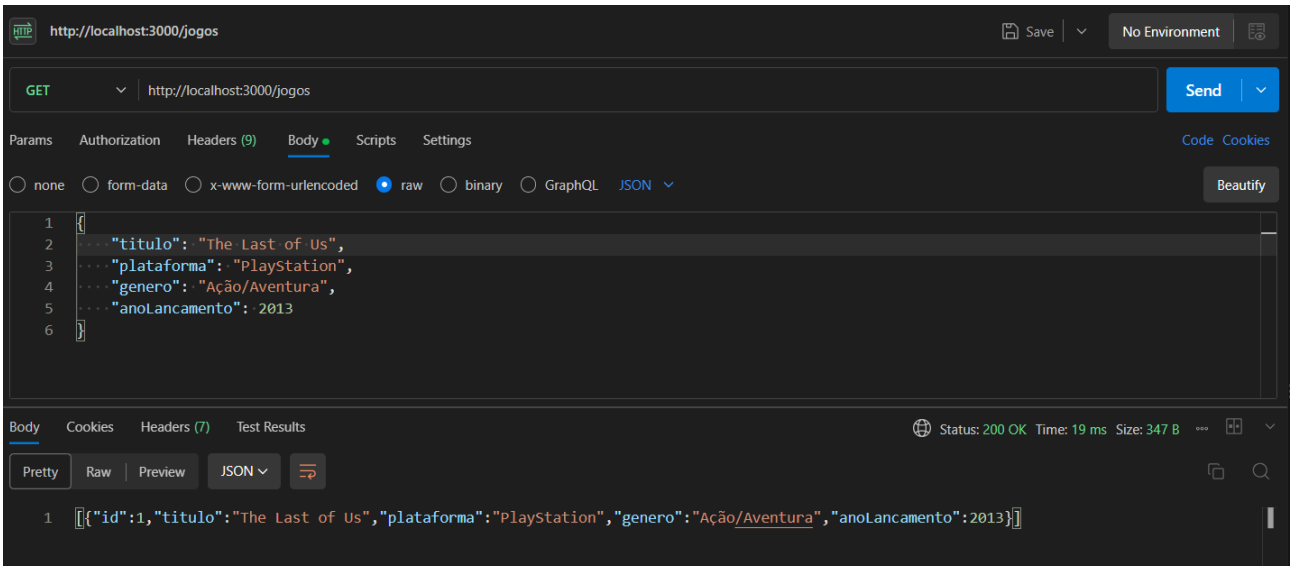


c) Implemente todas as rotas especificadas na tabela, utilizando os métodos HTTP adequados.

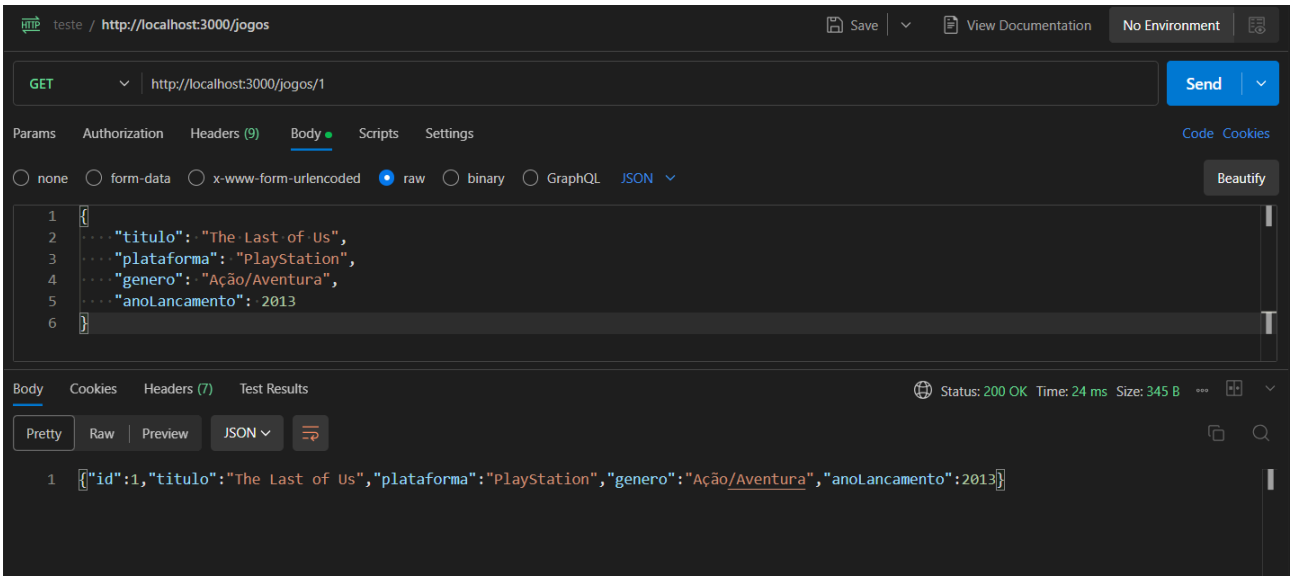


retorna o array vazio por enquanto.

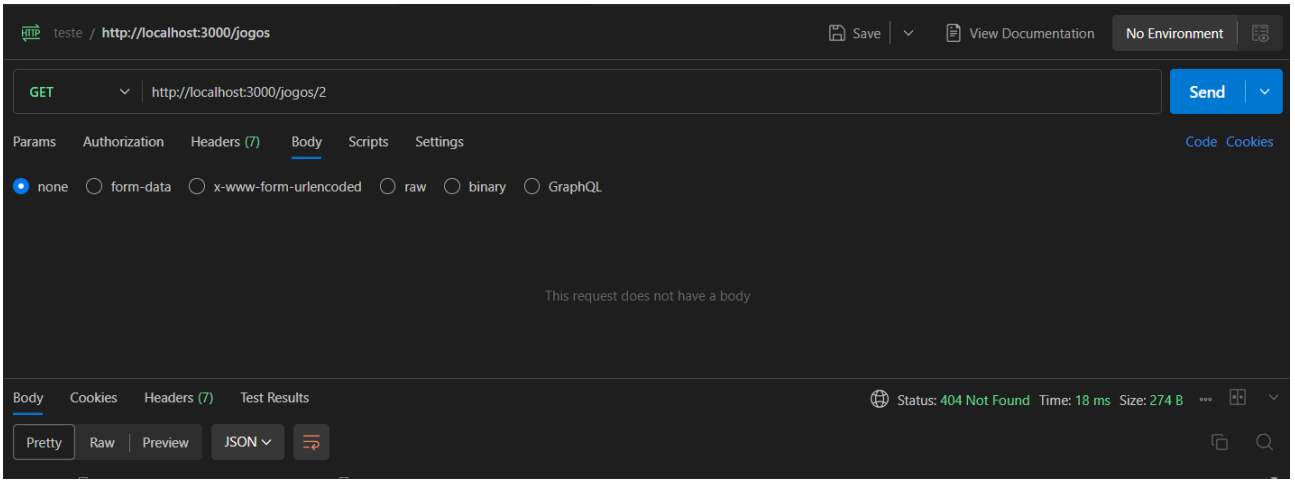
d) Na rota de cadastro, verifique se os campos titulo, plataforma, genero e anoLancamento foram informados. Caso contrário, retorne o status HTTP 400.



e) Ao consultar, atualizar ou excluir um jogo, verifique se o ID informado corresponde a um jogo cadastrado. Caso não exista, retorne o status HTTP 404.

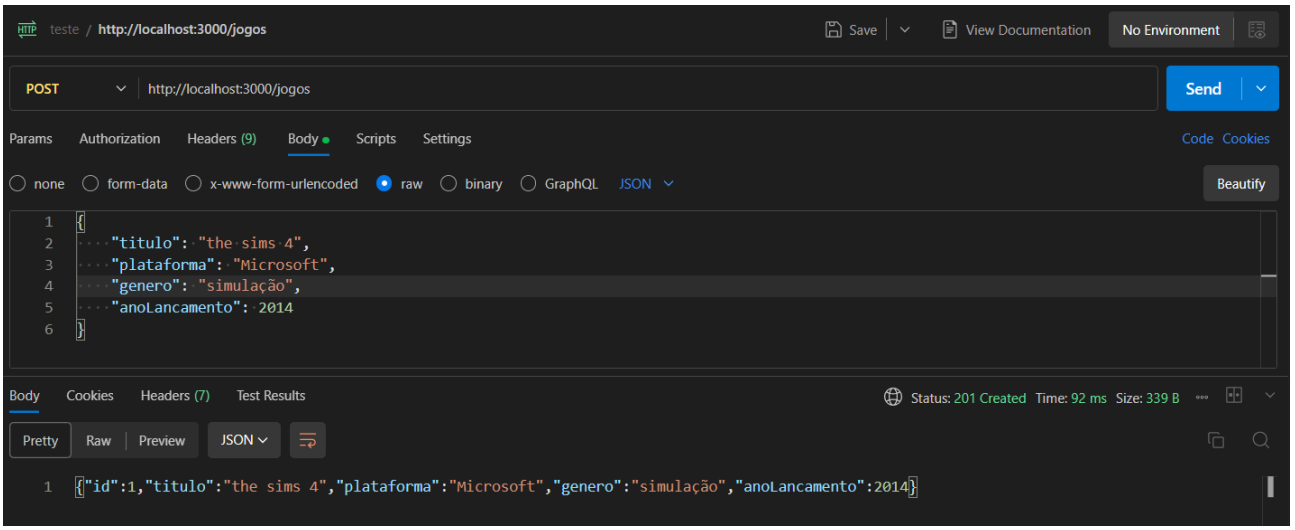


retorna 200 quando encontra.



retorna 404 quando não encontrado/existe.

f) Ao cadastrar um jogo, retorne o status HTTP 201. Nas demais operações realizadas com sucesso, retorne respostas adequadas ao resultado da operação.

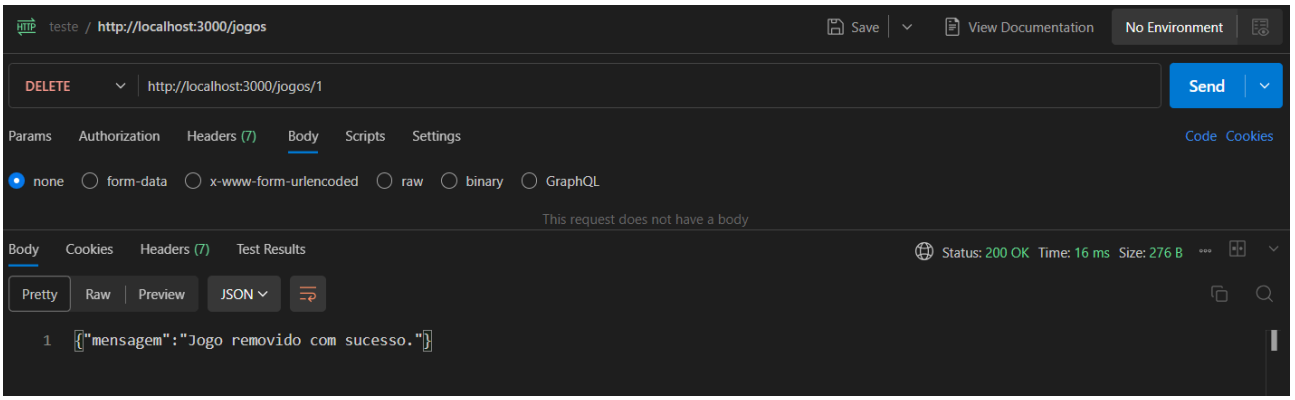


g) Crie um middleware que registre no console o método HTTP e a URL de cada requisição recebida.

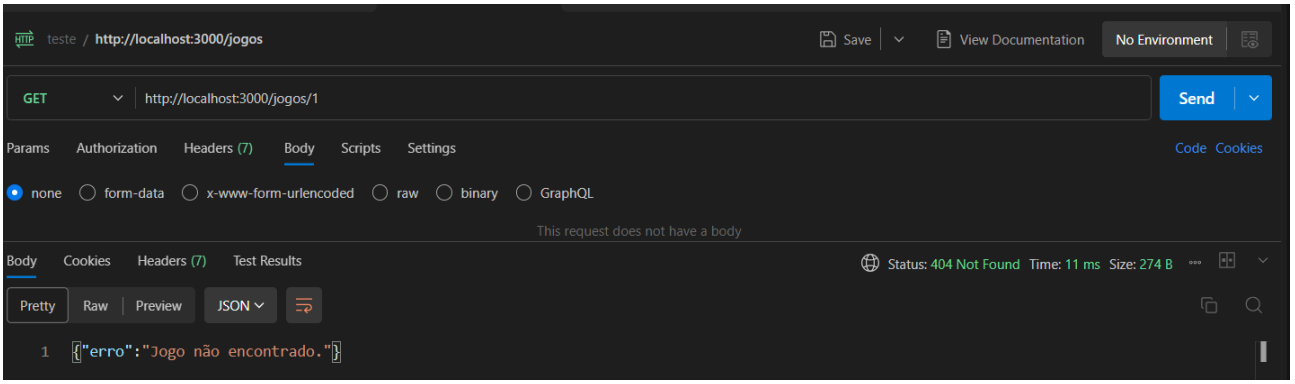
```
C:\Users\~Lucas Martins\Downloads\Atividade 4 andre\questao10>node index.js
Servidor de jogos rodando na porta 3000!
[GET] /jogos
```

dedurou o comando usado.

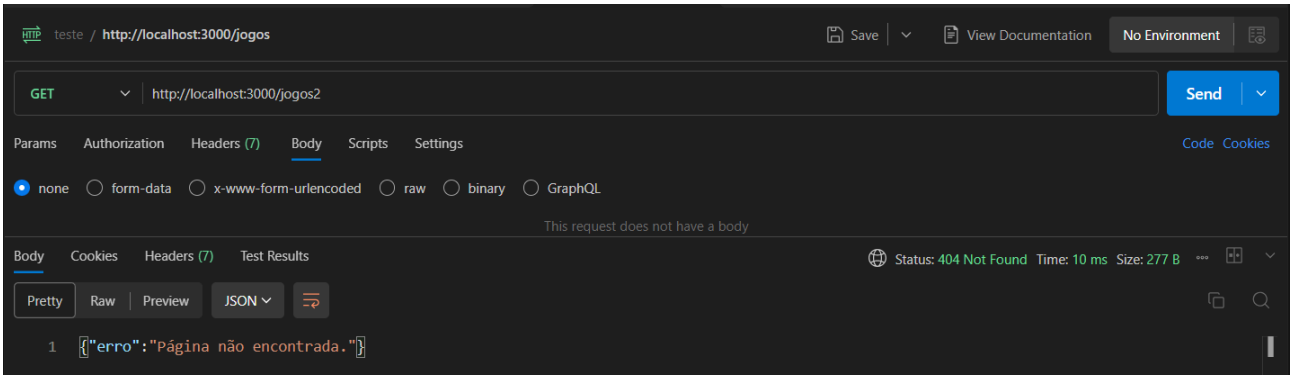
h) Implemente um tratamento para requisições direcionadas a rotas inexistentes, retornando uma mensagem em JSON e o status HTTP 404.



deletando…



o jogo não foi encontrado…



erro caso a rota seja inexistente.

i) Organize o código em arquivos separados para o servidor e para as rotas da API.
tá no github

Descrição da entrega: apresente o código-fonte dos arquivos desenvolvidos e demonstre o funcionamento de cada endpoint utilizando Postman, Insomnia ou outra ferramenta de teste de requisições HTTP.